
BIG DATA: Diseño y Arquitectura de Soluciones con Hadoop, Spark y R

Antonio Soto
CEO
asoto@solidq.com



Laboratorios

- Laboratorio 4 Trabajando con R en Notebooks Azure: Manejo de datos y visualización
- Laboratorio 5: Trabajo con SPARK
- Laboratorio 5: Trabajo con R en un cluster ML Services

Laboratorio 4 Azure Notebooks y R

- Importando datos
- Limpiando los datos
- Visualizando datos

3

Ejercicio 1 Azure Notebooks y carga de datos

Como primer paso vamos a cargar nuestro fichero `ventas_1000.csv` en la librería de Azure Notebooks para poder trabajar con él. Para ello crea un nuevo Notebook con kernel R, y desde el menú Data selecciona Upload. Tal y como indica la ventana, súbelo a `/library` para que nos quede persistente.

Ejercicio 2 Cargando datos

Una vez tenemos el fichero subido al entorno, incorpora el siguiente código a las celdas del notebook

```
#Leemos el CSV con read_csv del paquete readr (debemos de haber cargado el fichero)
```

```
library(readr)
```

```
Ventas <- read_csv("Ventas_1000.csv")
```

```
#Comprobamos las dimensiones del dataset
```

```
dim(Ventas)
```

```
#Inspeccionamos las primeras 6 observaciones (filas)
```

```
head(Ventas)
```

```
#Inspeccionamos las últimas 6 observaciones (filas)
```

```
tail(Ventas)
```

```
#observamos los nombres de las variables (columnas)
```

```
names(Ventas)
```

```
#Comprobamos la estructura del dataframe
```

```
str(Ventas)
```

```
#Repetimos lo anterior con glimpse() del paquete dplyr
```

```
library(dplyr)
```

```
glimpse(Ventas)
```

```
#vemos algunas características de las variables
```

```
summary(Ventas)
```

```
#Definimos Ventas2 quitando una serie de columnas que no nos interesan  
para el análisis
```

```
#Para ello primero definimos un vector con las posiciones de las columnas que  
vamos a quitar
```

```
#Quitaremos: Ticket, Linea, NombreCliente, UnidadMedida
```

```
indices <- c(1, 2, 6, 15)
```

```
#indexamos las variables con índice negativo para quitarlas
```

```
Ventas2 <- Ventas[, -indices]
```

```
#Vamos a separar Fecha, en Fecha y Hora
```

```
head(Ventas2$Fecha)
```

```
library(tidyr)
```

```
Ventas3 <- separate(Ventas2, Fecha, c("Fecha", "Hora"), sep = " ")
```

#Buscamos columnas que contengan en su nombre 'Fecha', en este caso se localiza con facilidad por el tamaño del dataset, pero en otros casos puede ser útil.

```
library(stringr)
cols_fecha <- str_detect(names(Ventas3), "Fecha")
```

#transformamos las columnas de fecha en tipo fecha, examinamos las primeras observaciones para ver en qué orden viene dada la fecha, y aplicamos la función correspondiente de lubridate.

```
head(Ventas3[,cols_fecha])
```

#vemos que es de tipo mm - dd - yyyy por lo que aplica mdy(). lapply() aplica la función que le pasamos como segundo argumento, sobre las columnas que le pasamos como primer argumento.

```
library(lubridate) Ventas3[,cols_fecha] <- lapply(Ventas3[,cols_fecha], mdy)
```

#volvemos a examinar para comprobar el cambio de tipo

```
head(Ventas3[,cols_fecha])
```

#vamos a crear una sola columna de 'Sexo' en lugar de tener una columna de hombre y otra de mujer

```
hombres <- Ventas3$Hombre == 1
mujeres <- Ventas3$Mujer == 1
Ventas3$Sexo[hombres] <- "Hombre"
Ventas3$Sexo[mujeres] <- "Mujer"
#examinamos el resultado
head(Ventas3$Sexo, n = 15)
```

```
#quitamos las columnas hombre y mujer
Ventas4 <- Ventas3[,c(-5,-6)]
#limpiamos la columna "ingresos anuales" que trae un error tipogr?fico en la
palabra 'm?s'
head(Ventas4$IngresosAnuales)
indices_error <- str_detect(Ventas4$IngresosAnuales, "de 70mil")
Ventas4$IngresosAnuales <- replace(Ventas4$IngresosAnuales, indices_error,
"M?s de 70mil")
head(Ventas4$IngresosAnuales)

#buscamos valores nulos 'NA' #vemos que efectivamente hay valores a NA ya
que any() devuelve 'TRUE'

any(is.na(Ventas4))

#buscamos donde estan esos NA

colSums(is.na(Ventas4))

#creamos un vector de ?ndices con los NA en Fecha (fecha y hora salieron
#de la misma variables con lo que los NA son en la misma observaci?n)

nulos <- which(is.na(Ventas4$Fecha))

#comprobamos esas observaciones con valores NA
Ventas4[nulos,]

#decidir qu? hacer con los nulos

#calculamos la edad con la fecha de nacimiento

Ventas4$Edad <- year(Sys.Date()) - year(Ventas4$FechaNacimiento)
```

```
head(Ventas4$Edad)
```

#ya no necesitamos la fecha de nacimiento, la quitamos del DF:

```
Carga Ventas5 <- Ventas4[, -5]
```

#buscamos outliers

```
boxplot(Ventas5$ImporteVenta)
```

#quitamos el caso mas extremo

```
outlier <- which(Ventas5$ImporteVenta > 100000)
```

```
Ventas5[outlier,]
```

```
Ventas5[outlier, "ImporteVenta"]
```

#quitamos el outlier

```
Ventas5 <- Ventas5[-outlier,] boxplot(Ventas5$ImporteVenta)
```

#quitamos valores negativos ya que solo queremos ver ventas, no devoluciones

```
negativos <- which(Ventas5$ImporteVenta < 0)
```

```
Ventas5 <- Ventas5[-negativos,]
```

```
boxplot(Ventas5$ImporteVenta~Ventas5$Familia)
```

#Exportamos el resultado a un nuevo csv

```
directorio <- "Ventas_1000_limpio.csv"
```

```
write_csv(Ventas4, directorio)
```

Ejercicio 3 Analizando datos

Vamos a partir ahora del nuevo fichero limpio que hemos creado y realizar algunas visualizaciones

```
#cargamos dataset limpio en lab08
```

```
Ventas5 <- read_csv("Ventas_1000_limpio.csv", locale = locale(encoding = "WINDOWS-1250"))
```

```
summary(Ventas5)
```

```
#vamos a pasar valores objeto de analisis a factores ()
```

```
factores <- c(3:9,13,14)
```

```
Ventas5[,factores] <- lapply(Ventas5[,factores], as.factor)
```

```
summary(Ventas5)
```

```
#exploremos un poco este cambio con levels() que devuelve los niveles del factor
```

```
levels(Ventas5$EstadoCivil)
```

```
levels(Ventas5$Sexo)
```

```
levels(Ventas5$m2_superficie)
```

```
#Vamos a categorizar las edades
```

```
Ventas5$Generacion <- cut(Ventas5$Edad,
```

```
breaks = c(Inf, 97, 77, 57, 37, 27, 7, -Inf),
```

```
labels = c("Generacion Grandiosa", "Generacion Silenciosa"
```

```
, "Baby Boomer", "Generacion X", "Generacion Y"
```

```
, "Millenial", "Tactil"),
```

```
right = FALSE)
```

```
#veamos como se distribuyen las ventas por familias
```

```
library(ggplot2)
```

```
ggplot(Ventas5, aes(x=Generacion, fill = ImporteVenta)) + geom_bar()
```

#montamos una tabla de contingencia de salario por estado civil

```
table(Ventas5$EstadoCivil, Ventas5$IngresosAnuales)
```

#vamos a ver esto de forma mas clara con un grafico de barras

```
ggplot(Ventas5, aes(x = IngresosAnuales, fill = EstadoCivil)) + geom_bar()
```

Laboratorio 5: SPARK

- Revisando parámetros de configuración
- Contando palabras con SPARK
- Dataframes
- Machine Learning
- SPARK Streaming

4

Ejercicio 1: Revisando parámetros de configuración

Los tres parámetros clave que se pueden utilizar para la configuración de Spark según los requisitos de la aplicación son **spark.executor.instances**, **spark.executor.cores** y **spark.executor.memory**. Un ejecutor es un proceso que se inicia para una aplicación Spark. Se ejecuta en el nodo de trabajo y es responsable de realizar las tareas de la aplicación. El número predeterminado de ejecutores y el tamaño de estos para cada clúster se calcula en función del número de nodos de trabajo y el tamaño de estos. Esta información se almacena en **spark-defaults.conf (/etc/spark2/2.6.2.25-1/0)** en los nodos principales del clúster. Los tres parámetros de configuración se pueden configurar en el nivel de clúster (para todas las aplicaciones que se ejecutan en el clúster) o se pueden especificar también para cada aplicación individual.

Es habitual que dependiendo del trabajo a realizar estas configuraciones se modifican para un trabajo en concreto en tiempo de ejecución, en lugar de modificarlas para todo el cluster. Vamos a ver como hacerlo desde un Notebook de Jupyter. Para ello:

- Iniciamos el entorno de JUPYTER bien desde el portal de Azure o bien directamente en el navegador: <https://NOMBRECLUSTER/JUPYTER>
- Creamos una nueva carpeta llamada Curso
- Dentro de la carpeta cargamos el fichero 00 Empezando con SPARK
- Insertamos una celda de tipo Code como primera celda y especificamos

```
%%configure
```

```
{"executorMemory": "3072M", "executorCores": 4, "numExecutors":10}
```

- Ejecutamos la celda y nos mostrará el cambio de configuración. Si quisiésemos cambiar la configuración una vez iniciada la sesión Spark, deberíamos de especificar el parámetro -f

Ejercicio 2: Cuenta palabras con SPARK

La segunda celda del Notebook nos muestra como cargar ficheros de texto y manipularlos desde Spark. El contexto que tenemos cargado de forma predeterminada utiliza YARN como modo de ejecución. Ejecutamos la segunda celda revisando las operaciones que va realizando en cada línea

Ejercicio 3: DataFrames

En este ejercicio, verás como trabajar con DataFrames importando datos desde ficheros, en este caso, csv. Verás también como aplicar transformaciones y filtros sobre esos datos, así como la creación de estructuras en memoria que te permitan después utilizar sentencias SELECT para la consulta de esos datos. Finalizarás el ejercicio viendo la interacción entre SPARKSQL y HIVE, almacenando los datos en una tabla y consultando tablas HIVE desde SPARK. Vete ejecutando las celdas desde la cabecera de DataFrames.

Ejercicio 4: Machine Learning

En este laboratorio veremos como crear, entrenar y consultar un modelo predictivo con Spark

Los datos siguientes muestran la temperatura objetivo y la temperatura real de algunos edificios con sistemas HVAC instalados. La columna System representa el identificador del sistema y la columna SystemAge, el número de años que lleva el sistema HVAC instalado en el edificio. Los datos de este tutorial se usan para predecir si un edificio será más cálido o frío en función de la temperatura objetivo, dados un identificador del sistema y la antigüedad del sistema

Cargaremos los datos del .csv. A través de una función que compara la temperatura real con la temperatura de destino indicaremos que si la temperatura real es mayor, el edificio está **cálido**, lo que viene indicado por el valor 1.0. De lo contrario, el edificio está **frío**, lo que se indica con el valor 0.0.

Generaremos un **pipeline** de Spark, utilizando un algoritmo de Regresión Logística.

Un **pipeline** es un flujo de trabajo completo que combina varios algoritmos de ML. Los pipelines definen las fases y el orden de un proceso de ML.

Un **transformador** es un algoritmo que transforma un elemento DataFrame en otro. Por ejemplo, un transformador de características puede leer una columna de un elemento DataFrame, asignarla a otra columna y generar un nuevo elemento DataFrame con la columna asignada anexada

Un **estimador** es una abstracción de algoritmos de aprendizaje y es responsable del ajuste o el aprendizaje en un conjunto de datos para producir un transformado

En nuestro caso generaremos un pipeline que consta de tres fases: tokenizer, hashingTF (ambos son transformadores) y lr:

1. En la primera fase, Tokenizer, se divide la columna de entrada SystemInfo (que consta de los valores de identificador y antigüedad del sistema) en una columna de salida words. Esta nueva columna words se agrega a DataFrame
2. En la segunda fase, HashingTF, se convierte la nueva columna words en vectores de característica. Esta nueva columna features se agrega al elemento DataFrame. Estas dos primeras fases son transformadores.
3. La tercera fase, LogisticRegression, es un estimador, por lo que la canalización llama al método LogisticRegression.fit() para generar un modelo LogisticRegressionModel.

Prepararemos después unos datos de prueba para probar el modelo y veremos sus predicciones.

Para todo ello, vete paso a paso a través del laboratorio 01 SPARK ML HVAC

Ejercicio 5: SPARK Streaming

Veremos en este ejercicio las bases de SPARK Streaming para crear aplicaciones que sean capaces de analizar datos en tiempo real a través de streams. Carga para ello el notebook 02 Bases de Spark Streaming y ejecuta las instrucciones como se te indican en el Notebook

En un escenario más real, lo habitual es utilizar un streaming estructurado que acabe generando tablas o ficheros para un consumo posterior por parte de ETL o directamente herramientas de BI. Para ello revisa el Notebook 03 Spark Streaming Estructurado

Laboratorio 6: R

- Consola R
- Importando datos y XDF
- Visualizando y Analizando Datos

5

En este laboratorio veremos como trabajar con un entorno de R instalado en un cluster Spark sobre Hdinsight. Recuerda que una de las diferencias entre este tipo de cluster y los cluster Spark o HDInsight es que aquí tenemos un nodo Edge al que debemos de conectarnos para interactuar, en el que estará instalado el entorno R y desde el que se configura todo ese entorno.

Ejercicio 1 Consola R

- Conecta por SSH al nodo `-ed` de tu cluster
- Ejecuta el comando **R** para entrar en el entorno
- Ahora ya podrías ejecutar sentencias R en el servidor Edge, por ejemplo `getwd()`

```
list.files(rxGetOption("sampleDataDir"))
```

Puedes consultar otras opciones como: `numCoresToUse`, `blocksPerRead`, `computeContext()`, `spark.executorCores`,.....

Ejercicio 2 Importando datos a R y gestionando XDF

En este ejercicio veremos las diferentes opciones que tenemos para cargar datos en estructuras R, y gestionar los XDF generados aplicando diferentes técnicas. Utiliza el fichero 01 Importando Datos y XDF para seguir paso a paso el ejercicio

Ejercicio 3 Visualizando y Analizando Datos

Una vez hemos visto como cargar y manejar los datos, el siguiente paso es dedicar un poco más de tiempo para visualizar y analizar esos datos, a través de una regresión logística para realizar una predicción. Sigue para ello el fichero 02 Visualizando y Analizando datos



www.solidq.com

info@solidq.com